

TRƯỜNG ĐẠI HỌC BÁCH KHOA – ĐẠI HỌC ĐÀ NẴNG
KHOA ĐIỆN TỬ - VIỄN THÔNG

BÁO CÁO PROJECT
MÔN HỌC: HỆ THỐNG NHÚNG

ĐỀ TÀI
Hệ thống chống trộm và định vị GPS
thông minh cho xe máy

Nhóm sinh viên thực hiện: Nhóm 04

01. Hồ Đức Duy	22DT2	106220050
02. Trần Duy Hiếu	22DT2	106220053
03. Nguyễn Đức Thịnh	22DT2	106220073
04. Hà Phước Huy	22DT4	106220136
05. Nguyễn Đức Phương Nam	22DT4	106220143

Người hướng dẫn: TS. Văn Phú Tuấn

Đà Nẵng, 02/05/2026

BẢNG PHÂN CÔNG CÔNG VIỆC THÀNH VIÊN

STT	HỌ VÀ TÊN	NHIỆM VỤ	ĐÓNG GÓP
01	Hồ Đức Duy	Vẽ lưu đồ thuật toán, class diagram, word chương 4	20%
02	Trần Duy Hiếu	Tìm hiểu Firebase, Platform IO, Android Studio, Code project, word chương 2, chương 3, hàn mạch	20%
03	Nguyễn Đức Thịnh	Tìm hiểu module; Thiết kế layout; sequence diagram	20%
04	Hà Phước Huy	Làm slide, mạch đồng	20%
05	Nguyễn Đắc Phương Nam	Word chương 1	20%

MỤC LỤC

BẢNG PHÂN CÔNG CÔNG VIỆC THÀNH VIÊN	1
MỤC LỤC	2
CHƯƠNG 1 – MỞ ĐẦU	3
1.1. Lý do chọn đề tài.....	3
1.2. Mục tiêu nghiên cứu.....	3
1.3. Những tính mới của hệ thống.....	4
CHƯƠNG 2 – GIỚI THIỆU HỆ THỐNG	6
2.1. Sơ đồ hệ thống.....	6
2.2. Danh mục linh kiện/thiết bị.....	6
2.3. Lưu đồ chức năng.....	8
CHƯƠNG 3 – KẾT QUẢ	11
3.1. Hình ảnh thực tế sản phẩm.....	11
3.2. Nhận xét kết quả (so với yêu cầu đặt ra, nêu các hạn chế)	14
3.3. Hướng phát triển	15
CHƯƠNG 4 – CHƯƠNG TRÌNH	17
TÀI LIỆU THAM KHẢO	30

CHƯƠNG 1 – MỞ ĐẦU

1.1. Lý do chọn đề tài

Trong bối cảnh hiện nay, xe máy vẫn là phương tiện di chuyển phổ biến tại Việt Nam do tính linh hoạt và chi phí phù hợp. Tuy nhiên, tình trạng mất cắp xe máy ngày càng diễn ra phức tạp, gây thiệt hại về tài sản và ảnh hưởng đến tâm lý người sử dụng. Các phương pháp bảo vệ truyền thống như khóa cơ hay khóa từ tuy có tác dụng nhất định nhưng vẫn tồn tại nhiều hạn chế, dễ bị vô hiệu hóa bởi các thủ đoạn tinh vi.

Bên cạnh đó, với sự phát triển của công nghệ IoT (Internet of Things), việc ứng dụng các hệ thống giám sát và định vị thông minh ngày càng trở nên khả thi và hiệu quả hơn. Các thiết bị tích hợp GPS cho phép xác định vị trí phương tiện theo thời gian thực, trong khi các cảm biến và cơ chế cảnh báo có thể giúp phát hiện sớm các hành vi xâm nhập hoặc tác động bất thường.

Xuất phát từ thực tế đó, đề tài “Hệ thống chống trộm và định vị GPS thông minh cho xe máy” được lựa chọn nhằm xây dựng một giải pháp kết hợp giữa phần cứng và phần mềm, cho phép theo dõi vị trí xe, phát hiện nguy cơ và đưa ra cảnh báo kịp thời. Đồng thời, đề tài cũng hướng đến việc tận dụng các nền tảng công nghệ hiện đại như ESP32, Firebase và ứng dụng di động để tạo ra một hệ thống có tính ứng dụng cao, phù hợp với nhu cầu thực tế.

1.2. Mục tiêu nghiên cứu

Mục tiêu của đề tài là xây dựng một hệ thống chống trộm và định vị GPS cho xe máy có khả năng hoạt động ổn định và đáp ứng được các nhu cầu cơ bản trong thực tế. Cụ thể, hệ thống hướng đến việc thu thập và hiển thị vị trí của phương tiện theo

thời gian thực thông qua module GPS, đồng thời cho phép người dùng theo dõi trực tiếp trên ứng dụng di động.

Bên cạnh đó, đề tài tập trung phát triển chức năng cảnh báo khi có tác động bất thường lên phương tiện, thông qua việc sử dụng cảm biến để phát hiện vật cản hoặc chuyển động. Khi xảy ra sự kiện, hệ thống có thể kích hoạt còi cảnh báo và gửi tín hiệu lên hệ thống trung gian để thông báo đến người dùng.

Ngoài ra, đề tài cũng hướng đến việc xây dựng một ứng dụng Android có khả năng tương tác với hệ thống, cho phép điều khiển từ xa các chức năng như bật/tắt chế độ cảnh báo (SOS), đồng thời hiển thị dữ liệu một cách trực quan. Việc sử dụng nền tảng IoT giúp đảm bảo khả năng đồng bộ dữ liệu giữa thiết bị và ứng dụng theo thời gian thực.

Thông qua quá trình thực hiện, đề tài cũng nhằm mục tiêu tìm hiểu và ứng dụng các công nghệ như ESP32, FreeRTOS, Firebase và lập trình ứng dụng di động, từ đó nâng cao khả năng thiết kế và triển khai một hệ thống nhúng có tính ứng dụng thực tế.

1.3. Những tính mới của hệ thống

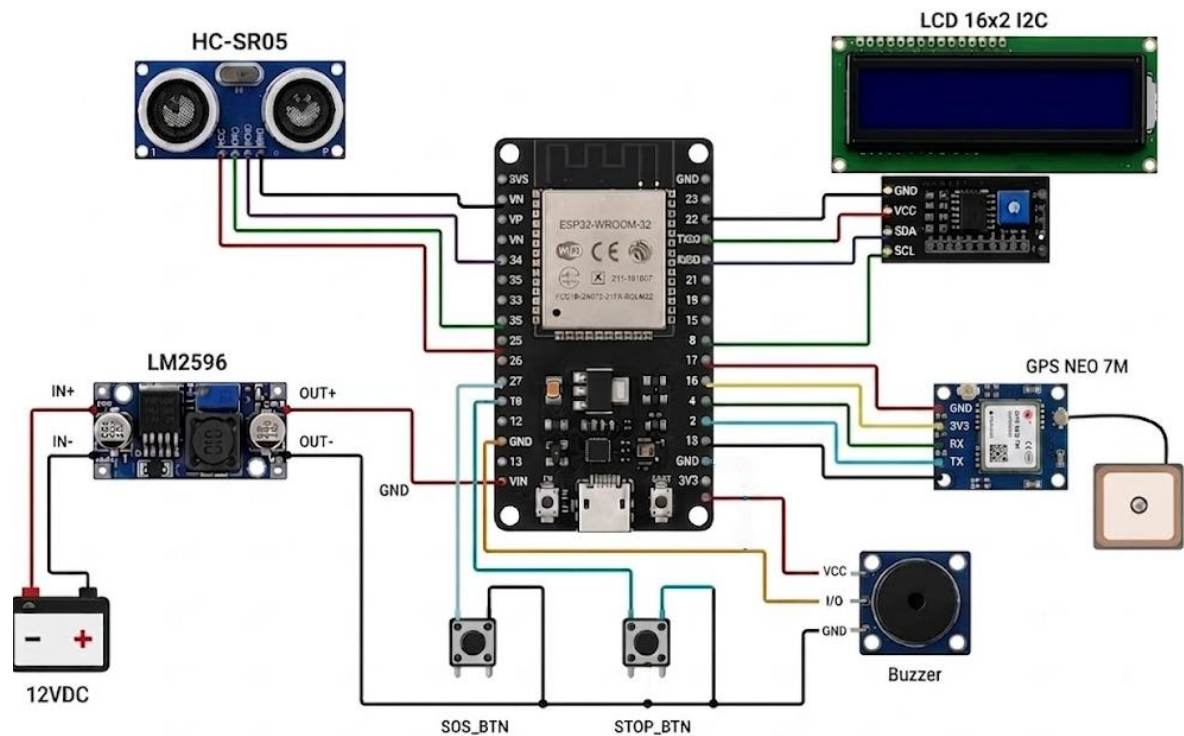
So với các hệ thống chống trộm truyền thống, đề tài này hướng đến việc tích hợp nhiều chức năng trong cùng một thiết bị, bao gồm định vị GPS, cảnh báo và điều khiển từ xa thông qua ứng dụng di động. Điểm khác biệt chính không nằm ở từng thành phần riêng lẻ mà ở cách kết hợp các công nghệ để tạo thành một hệ thống hoàn chỉnh.

Hệ thống cho phép theo dõi vị trí xe theo thời gian thực thông qua ứng dụng Android, đồng thời cập nhật và đồng bộ trạng thái giữa thiết bị và người dùng thông

qua nền tảng Firebase. Việc này giúp người dùng có thể nắm bắt tình trạng của xe một cách liên tục mà không cần thao tác trực tiếp.

CHƯƠNG 2 – GIỚI THIỆU HỆ THỐNG

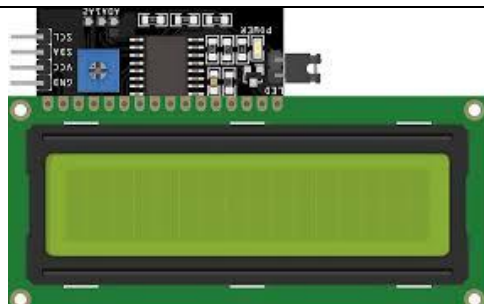
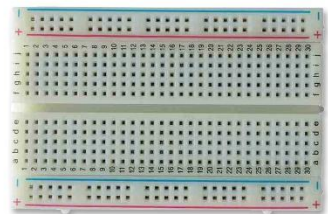
2.1. Sơ đồ hệ thống



2.2. Danh mục linh kiện/thiết bị

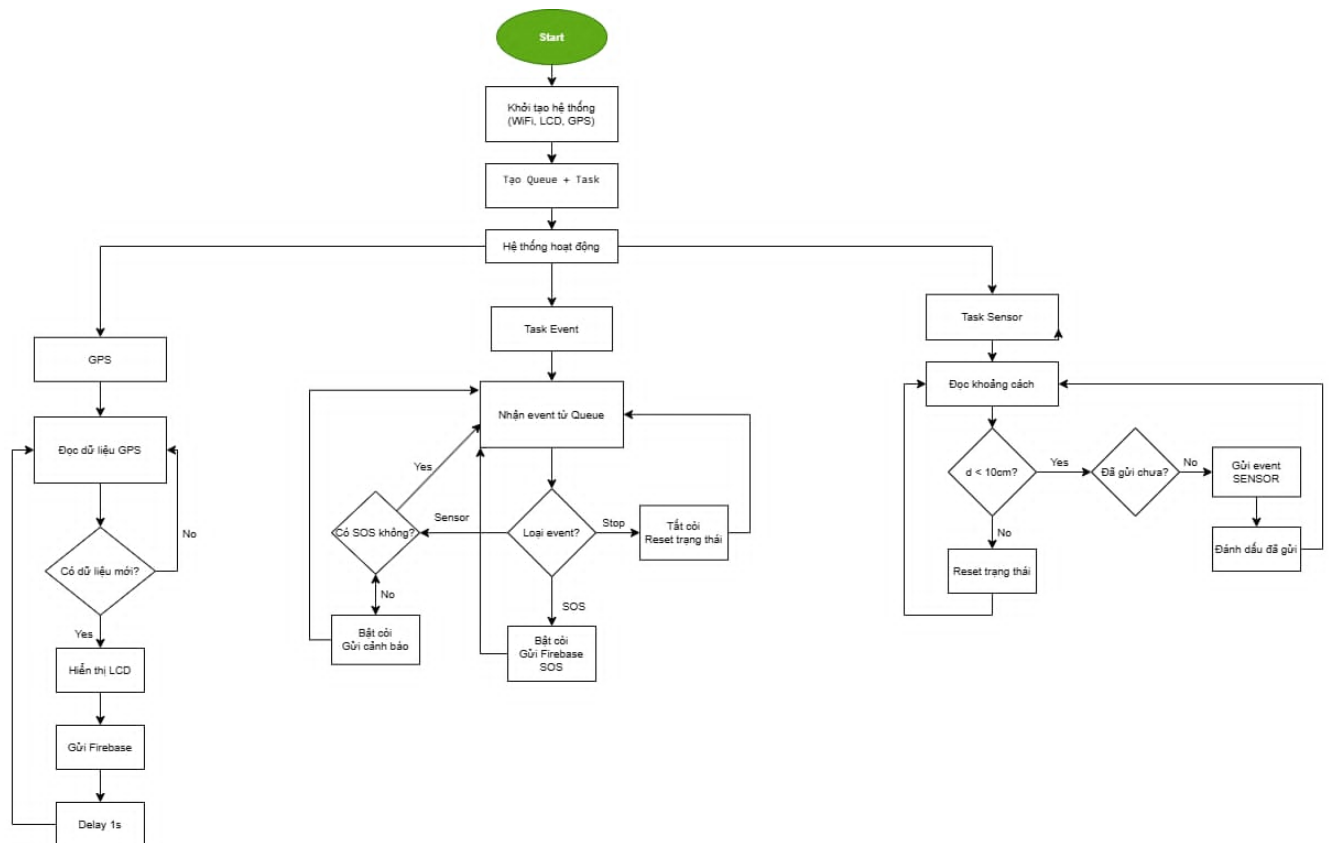
STT	Hình ảnh	Tên linh kiện	Chức năng	Giao thức / Kết nối
1		ESP32 DevKit	Vi điều khiển trung tâm, xử lý dữ liệu, kết nối WiFi,	WiFi, UART, GPIO, I2C

			điều khiển toàn hệ thống	
2		Module GPS (NEO-6M/7M)	Lấy vị trí (Latitude, Longitude) để tracking	UART (RX/TX)
3		Cảm biến siêu âm HC-SR04	Đo khoảng cách, phát hiện vật cản	Digital (TRIG/ECHO)
4		Buzzer	Phát âm thanh cảnh báo (SOS, khi có người tiếp cận)	GPIO (Digital Output)

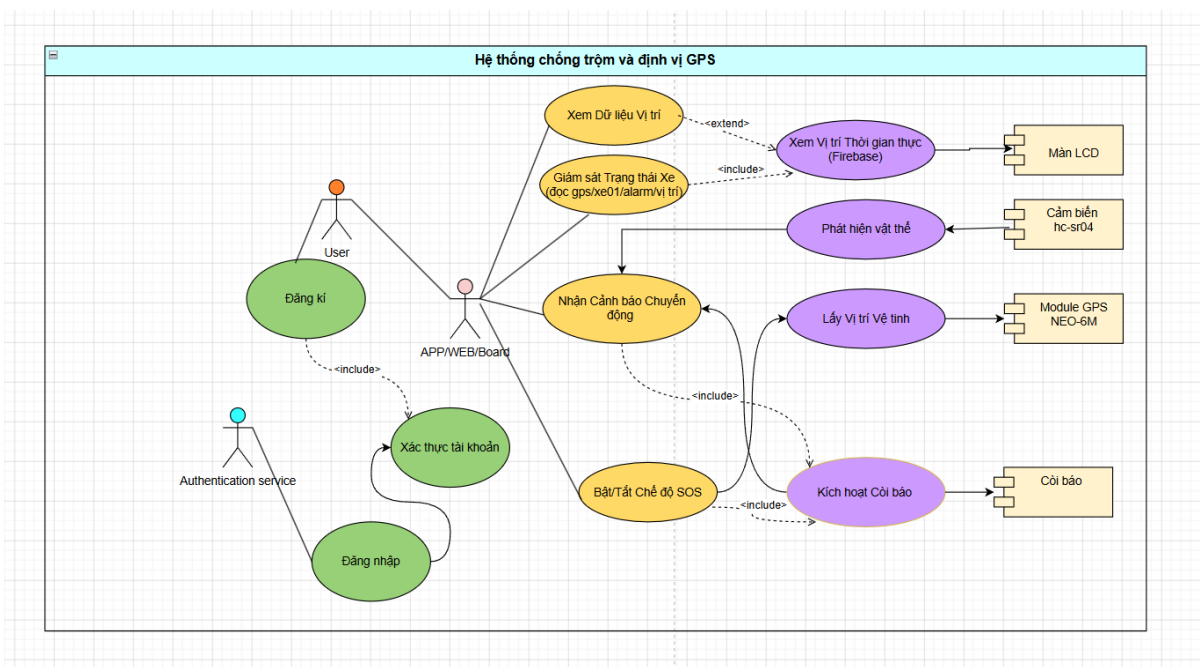
5		Nút nhấn (SOS, STOP)	Kích hoạt / tắt chế độ SOS	GPIO (Input Pull-up)
6		LCD 16x2 I2C	Hiển thị tọa độ GPS	I2C (SDA, SCL)
7		Breadboard	Lắp ráp mạch	-

2.3. Lưu đồ chức năng

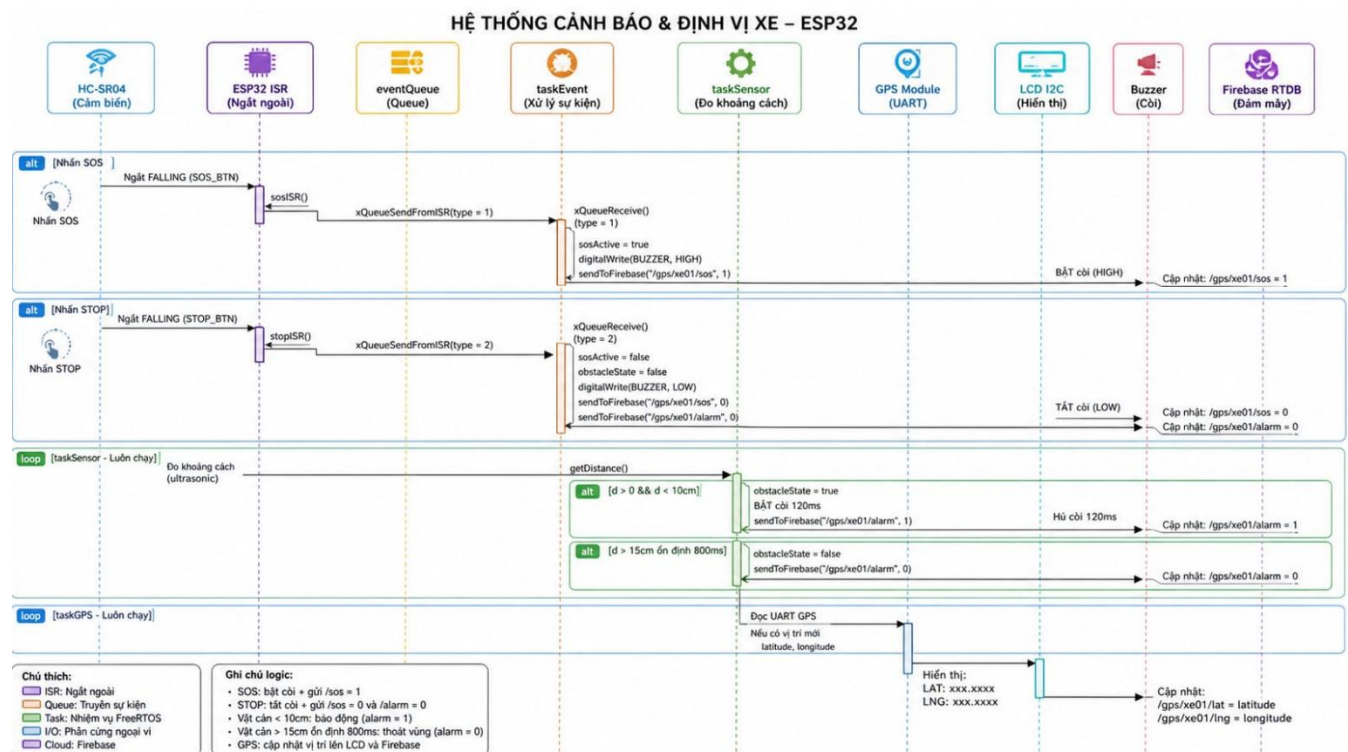
Lưu đồ giải thuật



User diagram



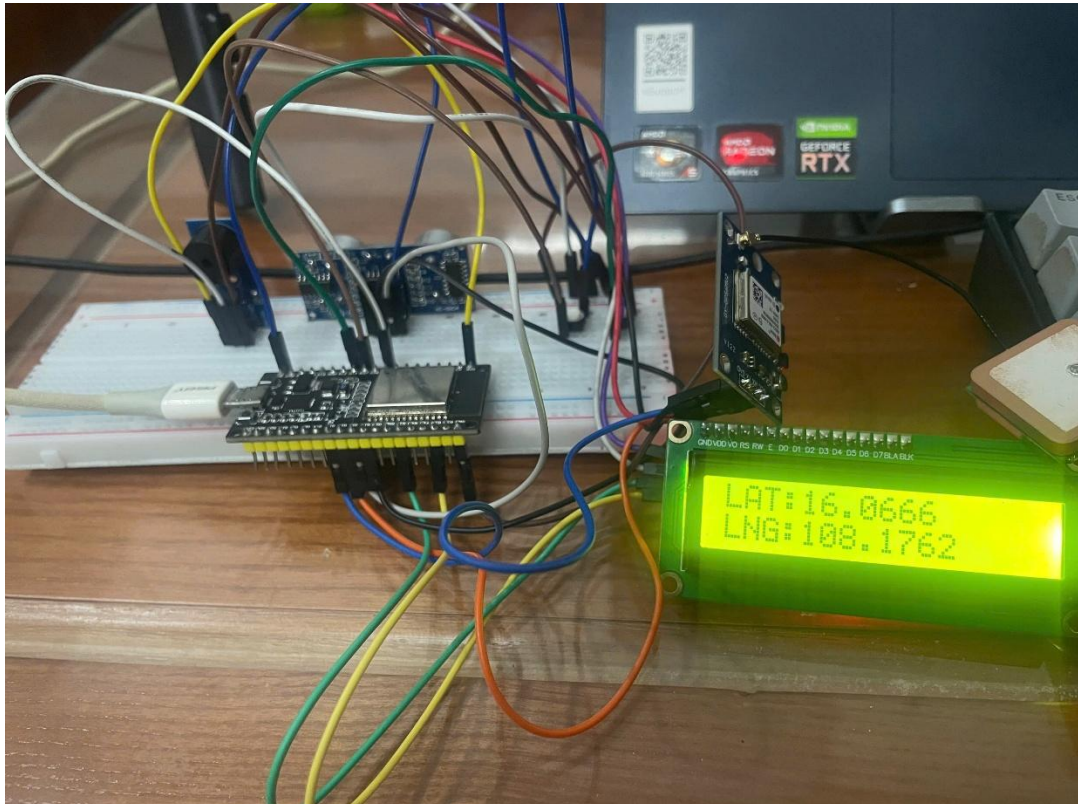
Sequence diagram



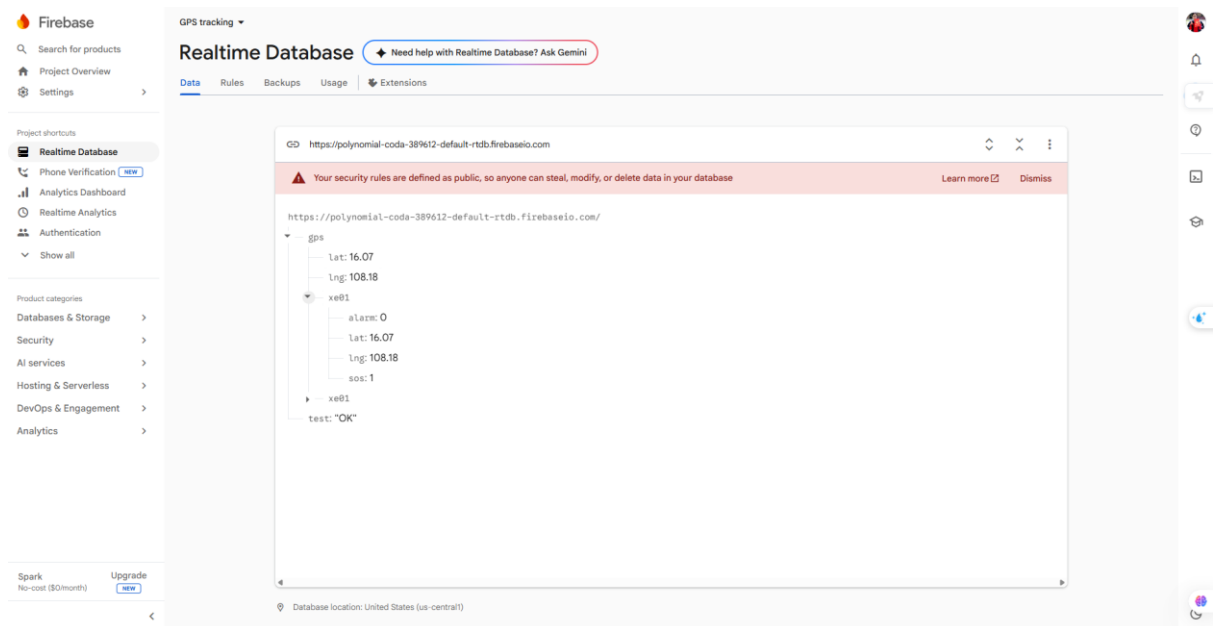
CHƯƠNG 3 – KẾT QUẢ

3.1. Hình ảnh thực tế sản phẩm

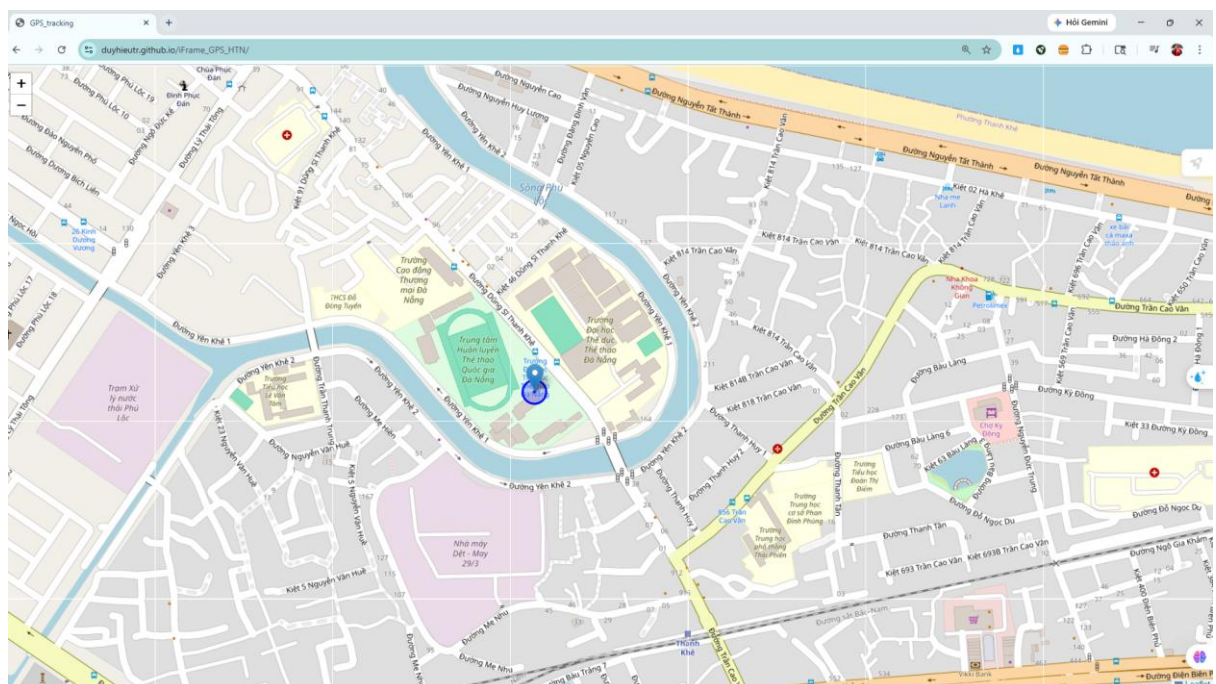
Hệ thống :



Cơ sở dữ liệu trên Firebase

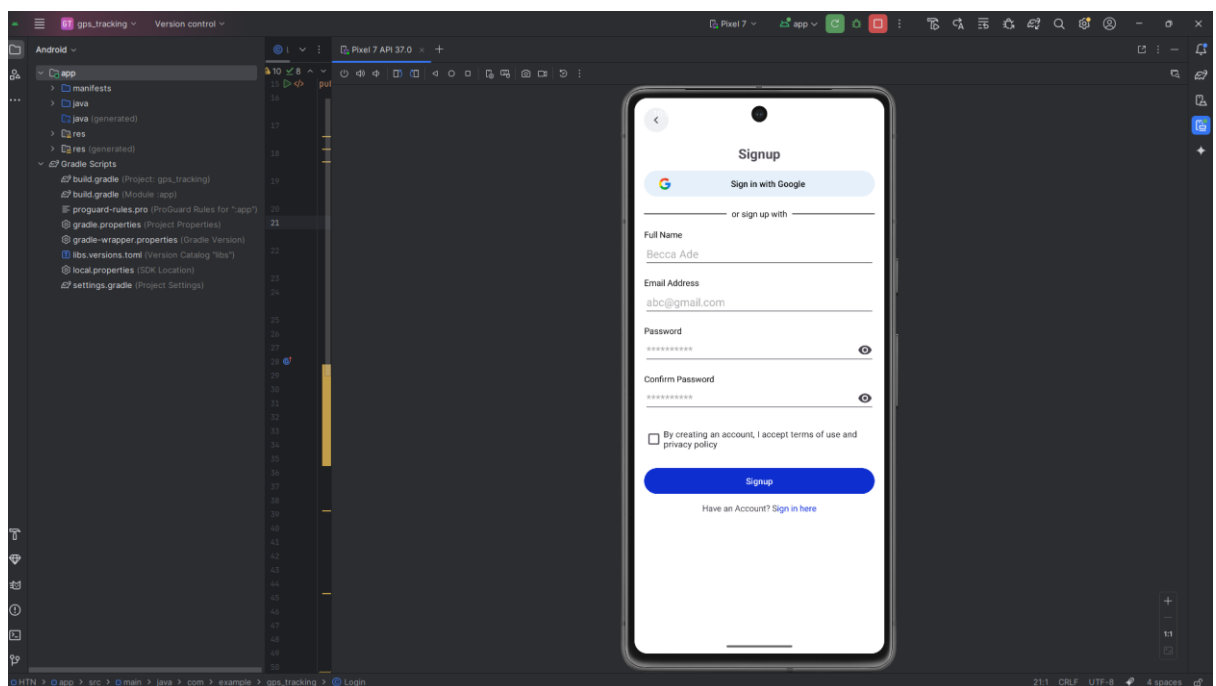
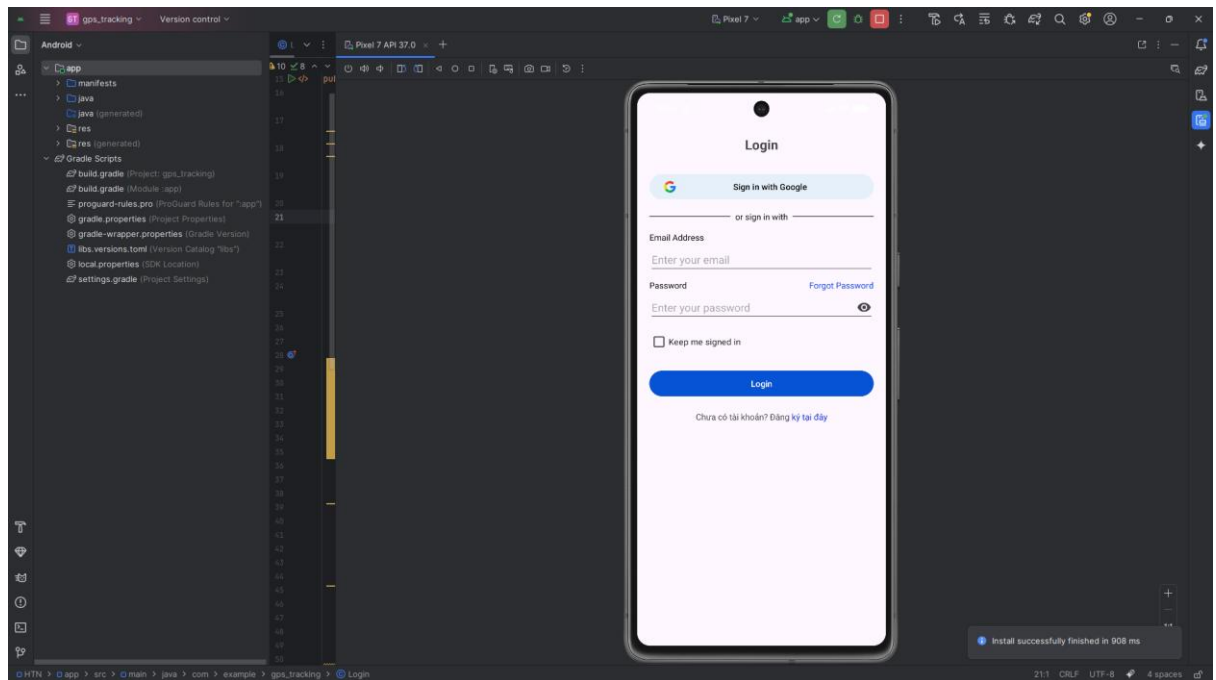


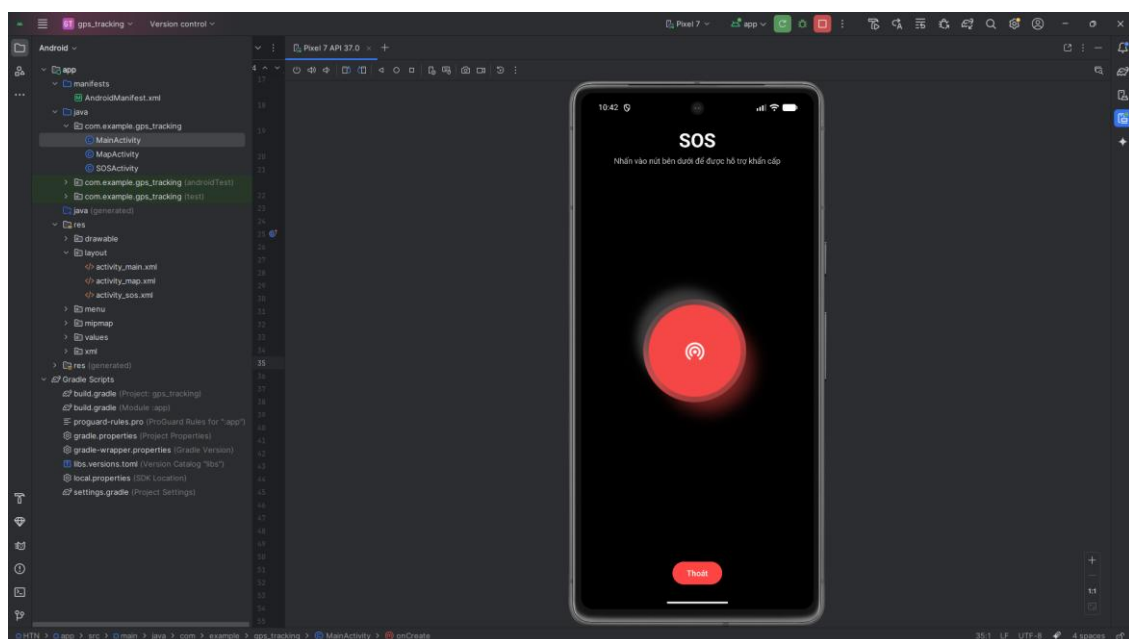
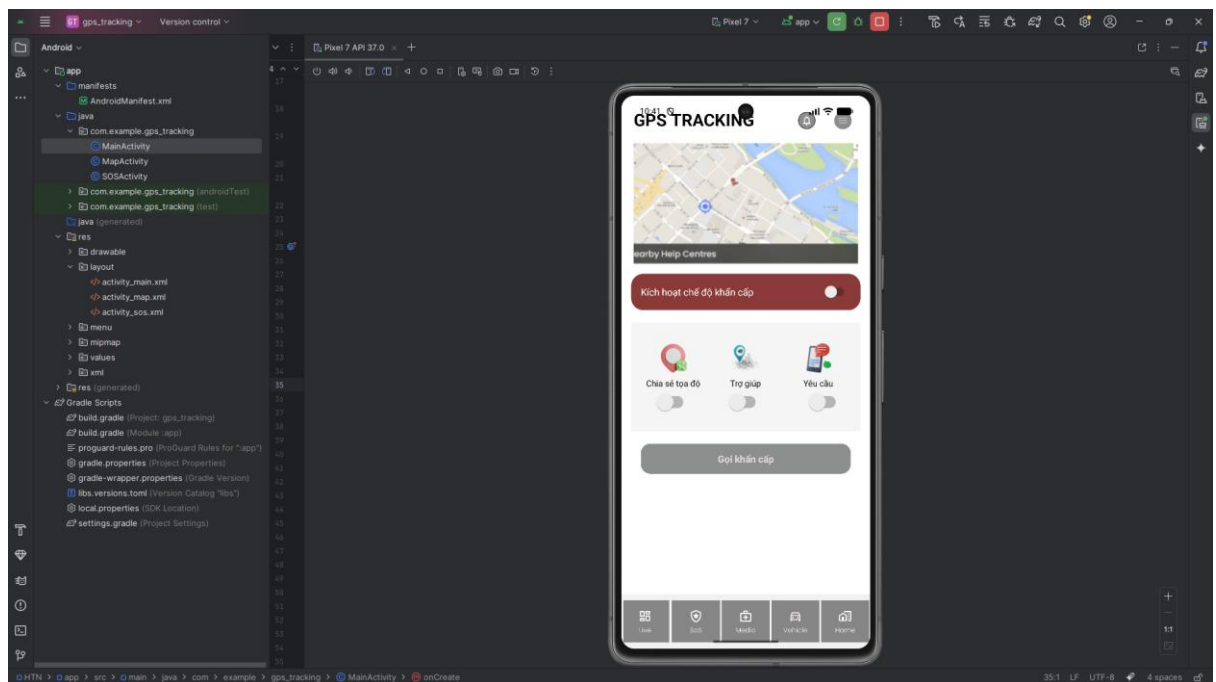
Web bản đồ hiển thị thời gian thực (deploy trên github)



App Android (lập trình trên android studio)

Giao diện





3.2. Nhận xét kết quả (so với yêu cầu đặt ra, nêu các hạn chế)

Nhìn chung, hệ thống đã thực hiện được các chức năng chính như ban đầu đề ra. ESP32 có thể đọc dữ liệu GPS để lấy vị trí, cảm biến HC-SR04 dùng để phát hiện vật cản và buzzer sẽ cảnh báo khi có nguy hiểm. Dữ liệu cũng được gửi lên Firebase để đồng bộ với ứng dụng Android. Ở phía app, đã làm được phần hiển thị bản đồ theo thời

gian thực, nên có thể theo dõi vị trí khá trực quan. Ngoài ra, chức năng SOS cũng đã hoạt động, có thể bật từ cả nút nhấn và trên app.

Về phần xử lý, hệ thống có sử dụng FreeRTOS để chia các công việc thành nhiều task như đọc cảm biến, xử lý sự kiện và cập nhật GPS nên chạy khá ổn định. Queue được dùng để truyền sự kiện giữa các task, còn interrupt giúp xử lý nút nhấn nhanh hơn. Một số chỗ có dùng Timer để xử lý thời gian, ví dụ như reset trạng thái cảm biến.

Tuy nhiên, vẫn còn khá nhiều điểm chưa hoàn thiện. Ở app Android, mới chủ yếu làm phần giao diện, nhiều nút bấm chưa xử lý logic phía sau nên chưa dùng được hết chức năng. Về phần cứng, cảm biến HC-SR04 đôi lúc còn bị nhiễu nên việc phát hiện vật cản chưa thật sự chính xác. Ngoài ra, hệ thống hiện tại vẫn phụ thuộc WiFi vì chưa sử dụng module SIM 4G A7680C, nên nếu không có mạng thì sẽ không gửi dữ liệu được. Một điểm nữa là trong code chưa dùng mutex để bảo vệ các biến dùng chung giữa các task, nên về lâu dài nếu mở rộng thêm chức năng thì có thể xảy ra lỗi.

Tóm lại, hệ thống đã chạy được và đúng hướng, nhưng vẫn cần cải thiện thêm để ổn định hơn và hoàn thiện đầy đủ các chức năng.

3.3. Hướng phát triển

Trong các bước tiếp theo, hệ thống cần được hoàn thiện theo hướng tăng tính ổn định và khả năng ứng dụng thực tế. Đối với ứng dụng Android, cần bổ sung phần xử lý logic cho các chức năng hiện có thay vì chỉ dừng ở giao diện, đồng thời cải thiện trải nghiệm người dùng như hiển thị trạng thái thiết bị rõ ràng hơn, cập nhật cảnh báo trực quan trên bản đồ và xử lý đồng bộ dữ liệu theo thời gian thực một cách chính xác hơn.

Ở phía phần cứng, vấn đề nhiễu của cảm biến HC-SR04 cần được xử lý tốt hơn thông qua các phương pháp lọc tín hiệu hoặc cân nhắc thay thế bằng loại cảm biến có độ ổn định cao hơn. Việc tích hợp thêm module SIM 4G A7680C cũng là một hướng cần thiết nhằm giúp hệ thống hoạt động độc lập với WiFi, đặc biệt trong các tình huống di chuyển hoặc môi trường không có kết nối mạng cố định.

Về mặt hệ thống, có thể cải tiến cách tổ chức chương trình bằng cách bổ sung cơ chế mutex để đảm bảo an toàn dữ liệu khi nhiều task cùng truy cập tài nguyên chung trong môi trường đa nhiệm. Đồng thời, việc mở rộng thêm các chức năng như lưu trữ lịch sử vị trí, gửi cảnh báo kèm thông tin vị trí hoặc tích hợp thêm cảm biến sẽ góp phần nâng cao tính hoàn chỉnh và khả năng mở rộng của hệ thống.

CHƯƠNG 4 – CHƯƠNG TRÌNH

MAIN.C

Chương trình chính có nhiệm vụ:

- Khai báo thư viện sử dụng (WiFi, GPS, LCD, HTTP...)
- Khai báo chân, biến toàn cục, queue
- Khởi tạo hệ thống (LCD, GPS, WiFi, ngắt...)
- Tạo các task FreeRTOS để xử lý song song:
 - + Đọc cảm biến
 - + Xử lý sự kiện
 - + Cập nhật GPS

```
#include <WiFi.h>
```

```
#include <Wire.h>
```

```
#include <TinyGPS++.h>
```

```
#include <HTTPClient.h>
```

```
#include <LiquidCrystal_I2C.h>
```

```
// WIFI
```

```
#define WIFI_SSID "Duy Hieu"
```

```
#define WIFI_PASS "28042004"
```

```
// FIREBASE
```

```
#define DATABASE_URL "https://polynomial-coda-389612-default-  
rtadb.firebaseio.com"
```

```
#define DATABASE_SECRET

"EyjryNGI1PMwWwDEBzSwFrkVxc8Fi03K4RY1KbWc"


// GPS

#define GPS_RX 16

#define GPS_TX 17


// HC-SR04

#define TRIG 5

#define ECHO 18


// BUTTON

#define SOS_BTN 27

#define STOP_BTN 14


// BUZZER

#define BUZZER 25


TinyGPSPlus gps;

HardwareSerial gpsSerial(1);

LiquidCrystal_I2C lcd(0x27, 16, 2);


// Biến GPS
```

```
float latitude = 0;

float longitude = 0;


// Trạng thái hệ thống

bool alarmActive = false;


// Struct sự kiện

typedef struct {

    int type; // 1 SOS, 2 STOP, 3 SENSOR

} Event_t;

QueueHandle_t eventQueue;
```

MAIN.C: TASK_SENSOR()

Task này có nhiệm vụ:

- + Đọc khoảng cách từ cảm biến siêu âm HC-SR04
- + Phát hiện vật cản trong khoảng < 10cm
- + Kích hoạt buzzer cảnh báo 1 lần
- + Gửi trạng thái cảnh báo lên Firebase
- + Reset trạng thái khi vật cản rời đi

```
void taskSensor(void *pv) {

    while (1) {
```

```
float d = getDistance();

uint32_t now = millis();

// Nếu phát hiện vật cản < 10cm
if (d > 0 && d < 10) {

    lastObstacleTime = now;

    if (!obstacleState) {

        obstacleState = true;

        digitalWrite(BUZZER, HIGH);

        delay(120);

        digitalWrite(BUZZER, LOW);

        sendToFirebase("/gps/xe01/alarm", 1); // Gửi cảnh báo

    }

}

if (obstacleState && d > 15) {

    if (now - lastObstacleTime > 800) {

        obstacleState = false;

        sendToFirebase("/gps/xe01/alarm", 0);

    }

}

if (!sosActive && !obstacleState) {

    digitalWrite(BUZZER, LOW);

}
```

```
    delay(120);  
  
    }  
  
}
```

MAIN.C: CHECK_SENSOR_TASK()

Chức năng:

Task này có nhiệm vụ:

- + Đọc khoảng cách từ cảm biến siêu âm HC-SR04
- + Nếu phát hiện vật cản < 10cm → gửi sự kiện vào queue
- + Tránh gửi lặp lại liên tục bằng biến alreadyTriggered

```
void taskSensor(void *pv) {  
  
    bool alreadyTriggered = false; // Cờ tránh gửi sự kiện lặp lại  
  
    while (1) {  
  
        float d = getDistance(); // Đọc khoảng cách từ cảm biến HC-SR04  
  
        // Nếu phát hiện vật cản trong khoảng < 10cm  
        if (d > 0 && d < 10) {  
  
            // Chỉ gửi 1 lần khi vừa phát hiện  
            if (!alreadyTriggered) {  
  
                Event_t e;
```

```
e.type = 3; // Gán loại sự kiện SENSOR

xQueueSend(eventQueue, &e, 0);

// Gửi sự kiện vào queue để taskEvent xử lý

alreadyTriggered = true; // Đánh dấu đã gửi
}

} else {

    alreadyTriggered = false;

    // Reset khi không còn vật cản để lần sau gửi lại

}

vTaskDelay(200 / portTICK_PERIOD_MS);

// Delay 200ms tránh đọc quá nhanh (giảm nhiễu)

}

}
```

MAIN.C: TASK_EVENT()

Chức năng:

- + Nhận sự kiện từ queue

- + Xử lý sự kiện SOS và STOP
- + Điều khiển buzzer
- + Gửi dữ liệu lên Firebase

```
void taskEvent(void *pv) {  
  
    Event_t e;  
  
    while (1) {  
  
        if (xQueueReceive(eventQueue, &e, portMAX_DELAY)) {  
  
            switch (e.type) {  
  
                case 1:  
  
                    sosActive = true;  
  
                    digitalWrite(BUZZER, HIGH);  
  
                    sendToFirebase("/gps/xe01/sos", 1);  
  
                    break;  
  
                case 2:  
  
                    sosActive = false;  
  
                    obstacleState = false;
```



```

digitalWrite(BUZZER, LOW);

sendToFirebase("/gps/xe01/sos", 0);

sendToFirebase("/gps/xe01/alarm", 0);

break;

}

}

}

}

```

MAIN.C: TASK_GPS()

Chức năng:

Đọc dữ liệu GPS

Cập nhật tọa độ

Hiển thị lên LCD

Gửi dữ liệu lên Firebase

```

void taskGPS(void *pv) {

while (1) {

while (gpsSerial.available())

```

```
gps.encode(gpsSerial.read());

if (gps.location.isUpdated()) {

    latitude = gps.location.lat();
    longitude = gps.location.lng();

    lcd.setCursor(0, 0);
    lcd.print("LAT:");
    lcd.print(latitude, 4);
    lcd.print(" ");

    lcd.setCursor(0, 1);
    lcd.print("LNG:");
    lcd.print(longitude, 4);
    lcd.print(" ");

    sendToFirebase("/gps/xe01/lat", latitude);
    sendToFirebase("/gps/xe01/lng", longitude);
}
delay(1000);
}
}
```

MAIN.C: INTERRUPT_HANDLER()

Chức năng:

- + Phát hiện nhấn nút SOS
- + Phát hiện nhấn nút STOP
- + Gửi sự kiện vào queue

```
void IRAM_ATTR sosISR() {
    Event_t e = {1};
    xQueueSendFromISR(eventQueue, &e, NULL);
}
```

```
void IRAM_ATTR stopISR() {
    Event_t e = {2};
    xQueueSendFromISR(eventQueue, &e, NULL);
}
```

MAIN.C: WIFI_FUNCTION()

Chức năng:

- + Hàm này dùng để kết nối WiFi

```
void connectWiFi() {
    WiFi.begin(WIFI_SSID, WIFI_PASS);
}
```

```
while (WiFi.status() != WL_CONNECTED) delay(300);  
  
}
```

MAIN.C: FIREBASE_FUNCTION()

Chức năng:

- + Hàm này dùng để gửi dữ liệu lên Firebase

```
void sendToFirebase(String path, float value) {  
  
    if (WiFi.status() != WL_CONNECTED) return;  
  
    HTTPClient http;  
  
    String url = String(DATABASE_URL) + path + ".json?auth=" +  
DATABASE_SECRET;  
  
    http.begin(url);  
  
    http.addHeader("Content-Type", "application/json");  
  
    http.PUT(String(value));  
  
    http.end();  
  
}
```

MAIN.C: SETUP()

Chức năng:

- + Khởi tạo hệ thống
- + Cấu hình GPIO
- + Khởi tạo LCD, GPS

- + Tạo queue và interrupt
- + Tạo các task FreeRTOS

```
void setup() {  
  
    Serial.begin(115200);  
  
    lcd.init();  
    lcd.backlight();  
  
    gpsSerial.begin(9600, SERIAL_8N1, GPS_RX, GPS_TX);  
  
    pinMode(TRIG, OUTPUT);  
    pinMode(ECHO, INPUT);  
  
    pinMode(SOS_BTN, INPUT_PULLUP);  
    pinMode(STOP_BTN, INPUT_PULLUP);  
  
    pinMode(BUZZER, OUTPUT);  
    digitalWrite(BUZZER, LOW);  
  
    eventQueue = xQueueCreate(10, sizeof(Event_t));  
  
    attachInterrupt(digitalPinToInterrupt(SOS_BTN), sosISR, FALLING);  
}
```

```
attachInterrupt(digitalPinToInterrupt(STOP_BTN), stopISR, FALLING);

connectWiFi();

xTaskCreatePinnedToCore(taskSensor, "sensor", 4096, NULL, 1, NULL, 1);
xTaskCreatePinnedToCore(taskEvent, "event", 4096, NULL, 2, NULL, 1);
xTaskCreatePinnedToCore(taskGPS, "gps", 4096, NULL, 1, NULL, 0);
}
```

TÀI LIỆU THAM KHẢO

- [1] “Mạch định vị toàn cầu dùng module GPS NEO-6M và module SIM 4G A7680C sử dụng kit WiFi NodeMCU ESP8266,” 2023. [Online]. Available: <https://www.youtube.com/watch?v=QQDtmFYWplA>
- [2] “GPS Tracking với E-Ra IoT Platform,” 2023. [Online]. Available: <https://www.youtube.com/watch?v=iJGvDuvo8kU>
Code: https://github.com/eoh-jsc/iFrame_GPS_Tracking
- [3] T.-D. Ngo, “ESP32 Firebase Google Realtime Database Part 01,” 2022. [Online]. Available: <https://www.youtube.com/watch?v=gkiP1Wz0mTI>
- [4] Google, “Android Developers Documentation,” 2026. [Online]. Available: <https://developer.android.com/develop?hl=vi>
- [5] Oracle, “Java Documentation,” 2026. [Online]. Available: <https://docs.oracle.com/en/java/>
- [6] Leaflet, “Leaflet.js Documentation,” 2026. [Online]. Available: <https://leafletjs.com/reference.html>
- [7] OpenAI, “ChatGPT,” 2026.

Link drive:

<https://drive.google.com/file/d/1mFQtvfu2bxVqTcubrMtEGMhojc2x7q-m/view?usp=sharing>

Link github :

https://github.com/duyhieutr/GPS_tracking_cuoiki_HTN_nhom4.git